

Стратегический план разработки (Roadmap) для создания саморазвивающейся операционной системы "piOS" на базе Rust, Verus и AIOS

1. Введение и архитектурная философия

Настоящий отчет представляет собой детальный план разработки (Todo List) и архитектурное обоснование для создания операционной системы нового поколения, условно именуемой "**piOS**". Данная система объединяет в себе четыре передовые парадигмы, которые до настоящего момента развивались изолированно: безопасное системное программирование на **Rust**, формальная верификация корректности кода с использованием инструментария **Verus**, архитектура **AI-Native (AIOS)**, где искусственный интеллект является не приложением, а системным ресурсом, и концепция **самонаписания кода (Self-coding)**, позволяющая системе автономно развиваться и оптимизировать собственные компоненты.

Цель разработки — разрешить фундаментальное противоречие между надежностью (Reliability) и адаптивностью (Adaptability). Традиционные операционные системы, такие как Linux или Windows, полагаются на огромные базы унаследованного кода на C/C++, где безопасность обеспечивается дисциплиной разработки и тестированием, а не математическим доказательством. В предлагаемой архитектуре доверенная вычислительная база (Trusted Computing Base, TCB) математически верифицируется, а гибкость обеспечивается агентами ИИ, работающими в изолированных средах WebAssembly (WASM), способными генерировать, верифицировать и внедрять новый код без остановки системы.

1.1 Архитектурные столпы

1. **Верифицированное ядро (Verified Microkernel):** Основано на Rust и Verus. В отличие от проектов типа Redox или Theseus, здесь ставится задача не просто использовать memory-safe язык, но и математически доказать отсутствие логических ошибок (deadlocks, race conditions, integer overflows) и корректность выполнения спецификаций. Используется методология "framekernel", где верифицируется только критически важный код, управляющий unsafe операциями.
2. **WebAssembly Component Model:** Все драйверы, системные сервисы и пользовательские приложения исполняются в среде WASM. Это обеспечивает аппаратную независимость и строгую изоляцию ("песочницу"). Компонентная модель (Component Model) позволяет динамически связывать модули, что критически важно для горячей замены кода при самообновлении.
3. **AI-Native Orchestration (AIOS):** Языковые модели (LLM) интегрированы в ядро

через интерфейс WASI-NN. Операционная система предоставляет "Шину Интеллекта" (Intelligence Bus), где агенты управляют планированием процессов, распределением памяти и оптимизацией ввода-вывода на основе семантического понимания задач пользователя, а не эвристик.

- 4. **Цикл Автопоэзиса (Self-Coding Loop):** Система включает в себя встроенный компилятор (на базе Cranelift/Winch) и верификатор Verus как сервисы. ИИ может сгенерировать улучшенную версию драйвера, автоматически отправить её на верификацию, и, в случае успеха доказательства безопасности, ядро произведет горячую замену компонента.

1.2 Структура плана работ

План разбит на 50 рабочих пакетов (Work Packages, WP). Каждый пакет оценивается в **80 человеко-часов** (стандартный двухнедельный спринт для одного высококвалифицированного инженера). Общая трудоемкость составляет 4000 человеко-часов, что соответствует примерно году работы команды из 2-3 ключевых разработчиков. Каждый пункт содержит не просто задачу, но и критерии верификации (что именно мы доказываем), необходимые исследования и ожидаемые артефакты.

2. Фаза I: Формально верифицированное микроядро (The Verified Core)

Цель фазы: Создать минимально необходимое, математически доказанное основание (TCB), способное загружаться, управлять физической памятью и обеспечивать базовую изоляцию процессов. **Технологический стек:** Rust (no_std), Verus, QEMU, UEFI, x86_64/AArch64 assembly.

В этой фазе закладывается фундамент безопасности. Мы не просто пишем код, мы пишем спецификации поведения системы и доказательства того, что код им соответствует. Это самый трудоемкий этап, требующий высокой квалификации в формальных методах.

Раздел 1.1: Загрузка и управление памятью (Memory Management)

Этот раздел критичен для безопасности. Ошибки здесь ведут к компрометации всей системы. Verus позволяет доказать, что ни один физический адрес памяти не используется двумя владельцами одновременно (aliasing) без явного разрешения.

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации Verus
WP-001	Настройка среды Verus и UEFI Boot Stub	Описание: Настройка тулчейна (Rust nightly + Verus), интеграция с системой сборки. Разработка загрузочного "стаба" на uefi-rs, который переводит процессор в Long Mode и передает управление ядру.

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации Verus
		Критерии верификации: Необходимо формализовать контракт передачи управления (handoff). Доказать, что карта памяти (Memory Map), полученная от UEFI, корректно преобразуется во внутренние структуры ядра без потери данных о зарезервированных регионах. Это устанавливает "Корень доверия" (Root of Trust) для аллокатора памяти. Артефакт: Загружаемый образ, отчет Verus о проверке инициализации.
WP-002	Верифицированный аллокатор физических страниц (PMM)	Описание: Реализация аллокатора (например, bitmap или stack-based). Верификация: В Verus создается абстрактная модель "Множество свободных страниц". Доказательство должно гарантировать инвариант: "Страница P, возвращенная функцией alloc(), удалена из множества свободных, и никто другой не владеет ссылкой на P". Это предотвращает use-after-free на уровне физических фреймов. Сложность: Моделирование битовых операций в SMT-солвере может быть нетривиальным, требуется использование библиотеки vstd::bits.
WP-003	Формальная спецификация таблиц страниц (Page Tables)	Описание: Это чисто теоретическая задача по написанию "Призрачного кода" (Ghost code) в Verus. Необходимо описать рекурсивную структуру

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации Verus
		таблиц страниц (PML4, PDPT, PD, PT) для x86-64. Верификация: Определить предикат <code>valid_mapping(virt, phys, perms)</code> . Это будет "золотой стандарт", с которым будет сверяться реализация. Спецификация должна исключать возможность создания циклов в таблицах страниц или отображения защищенных регионов ядра в user-space.
WP-004	Верифицированный манипулятор таблиц (Mapper)	Описание: Реализация функций <code>map</code> , <code>unmap</code> , <code>protect</code> . Это код, работающий с <code>unsafe</code> указателями. Верификация: Доказать, что реализация является <i>уточнением</i> (refinement) спецификации из WP-003. Критически важно доказать отсутствие TLB-incoherency (рассинхронизации кэша трансляции адресов) — доказательство должно требовать вызова <code>invlpg</code> при изменении маппинга. Asterinas предоставляет примеры доказательства корректности навигации по дереву таблиц.
WP-005	Ядерный аллокатор кучи (Verified Slab Allocator)	Описание: Реализация <code>GlobalAlloc</code> для нужд ядра (динамические структуры данных). Верификация: Доказать отсутствие переполнения буфера и корректность метаданных блоков. В отличие от стандартного Rust, Verus позволяет доказать корректность арифметики указателей внутри аллокатора. Это устраняет

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации Verus
		класс уязвимостей heap corruption.

Раздел 1.2: Примитивы синхронизации и Планирование

Многозадачность вводит недетерминизм, который сложно тестировать, но можно верифицировать.

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации Verus
WP-006	Верифицированный Спинлок (Spinlock)	Описание: Базовый примитив синхронизации. Rust обеспечивает безопасность типов, но не гарантирует отсутствие дедлоков (deadlocks). Верификация: Использование механики "токенов" (tokens) в Verus для моделирования права на захват. Доказательство того, что поток, удерживающий спинлок, не прерывается (отключение прерываний), и что порядок захвата (lock ordering) соблюдается для предотвращения дедлоков.
WP-007	Абстракция "Поток" (Thread) и Переключение контекста	Описание: Структура TCB (Thread Control Block) и ассемблерный код переключения стеков. Верификация: Ассемблер верифицировать сложно, поэтому верифицируется Rust-обертка. Доказывается, что при вызове switch_context сохраняются инварианты регистров (callee-saved). Вводится абстрактное состояние "CPU State", и доказывается переход между состояниями потоков (Running -> Ready -> Suspended).
WP-008	Формально верифицированный Планировщик (Scheduler)	Описание: Реализация алгоритма Round-Robin или Priority Queue. Верификация:

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации Verus
		Доказательство свойств "Живучести" (Liveness): каждый поток в состоянии Ready <i>гарантированно</i> получит квант времени. Доказательство "Безопасности" (Safety): планировщик никогда не передаст управление на неинициализированный или уничтоженный поток. Использование методологии CONVEROS для проверки моделей конкурентности.
WP-009	Механизм IPC (Message Passing)	Описание: Синхронная передача сообщений между процессами (Rendezvous). Это основа микроядерной архитектуры. Верификация: Доказательство атомарности передачи владения данными. Если Процесс А передает буфер Процессу Б, Verus должен доказать, что Процесс А теряет доступ к этому буферу в тот же момент, когда Процесс Б его получает (Linear Types logic). Это исключает гонки данных (data races) на уровне архитектуры.
WP-010	Драйвер контроллера прерываний (APIC/GIC)	Описание: Настройка маршрутизации прерываний. Верификация: Моделирование контроллера как конечного автомата (State Machine). Доказательство того, что каждое аппаратное прерывание корректно маскируется и демаскируется (Ack/EOI), предотвращая шторм прерываний или их потерю.

3. Фаза II: WebAssembly User Space и Модель

Драйверов

Цель фазы: Реализовать пространство пользователя, где вместо нативных бинарных файлов (ELF/PE) используется WebAssembly. Это ключевой момент для реализации "самонаписания" (Self-coding) в будущем, так как WASM обеспечивает безопасную песочницу для сгенерированного ИИ кода. **Технологический стек:** Wasmtime (no_std fork), WASI (WebAssembly System Interface), WIT (WASM Interface Types).

Раздел 2.1: Интеграция WASM Runtime в Ядро

Запуск WASM в кольце защиты Ring 0 (режим ядра) позволяет избежать накладных расходов на переключение контекста, сохраняя при этом изоляцию memory-safety.

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации
WP-011	Портирование WASM Runtime (no_std)	Описание: Адаптация легковесного рантайма (например, wasmi для начала, позже wasmtime) для работы без стандартной библиотеки (std). Замена аллокаций на ядерный GlobalAlloc (WP-005). Инсайт: Это позволяет превратить ОС в "Unikernel", где приложения работают в одном адресном пространстве, но изолированы виртуальной машиной WASM.
WP-012	Верификация песочницы памяти WASM	Описание: WASM гарантирует безопасность памяти через проверку границ (bounds checking). Верификация: Необходимо верифицировать "клей" (glue code) между рантаймом WASM и памятью ядра. Доказать в Verus, что функция memory.read/write рантайма никогда не выйдет за пределы выделенного региона WasmMemory, даже если байт-код скомпрометирован. Это защита от побега из песочницы (sandbox escape).
WP-013	Реализация WASI-Core (Syscalls)	Описание: Реализация стандартов wasi-io,

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации
		wasi-filesystem поверх примитивов ядра (IPC из WP-009). Верификация: Проверка соответствия спецификации WASI. Доказательство того, что файловый дескриптор, переданный в WASI-вызов, соответствует валидному объекту ядра и правам доступа (Capabilities).
WP-014	Динамический Линковщик Компонентов (Linker)	Описание: Реализация механизма загрузки и связывания WASM-компонентов согласно Component Model. Это позволяет "подключать" драйверы как плагины. Важность: Это основа для Self-coding. Система сможет выгрузить старый компонент драйвера и загрузить новый, сгенерированный ИИ, на лету.
WP-015	Система Капабилити (Capabilities System)	Описание: Реализация таблицы ресурсов (Resource Table) для каждого компонента. Доступ к железу (портам ввода-вывода, памяти DMA) выдается только через типизированные хендлы. Верификация: Доказать невозможность подделки хендла (handle forging). Компонент не может создать хендл "из воздуха", он должен получить его от родителя или ядра.

Раздел 2.2: Инфраструктура Драйверов на WASM

Драйверы в riOS — это WASM-компоненты, которые общаются с ядром через строго определенные интерфейсы.

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации
WP-016	Драйвер VirtIO-Blk (WASM)	Описание: Реализация драйвера блочного

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации
		устройства (диска) на Rust, компилируемого в WASM. Верификация: Доказательство корректности работы с кольцевыми буферами (Virtqueue). Инварианты: индексы avail и used колец всегда находятся в допустимых пределах, предотвращая переполнение буфера при обмене данными с гипервизором/железом.
WP-017	Драйвер VirtIO-Net (WASM)	Описание: Сетевой драйвер. Верификация: Формализация обработки пакетов. Доказательство отсутствия утечек памяти при передаче пакетов (packet buffers) между сетевой картой и сетевым стекком.
WP-018	PCI Bus Enumerator	Описание: Логика сканирования шины PCI и загрузки соответствующих WASM-драйверов. Верификация: Доказательство корректности парсинга конфигурационного пространства PCI. Гарантия того, что драйвер загружается только для устройства с совпадающим VendorID/DeviceID.
WP-019	Абстракция DMA (Safe DMA)	Описание: Безопасный интерфейс для Direct Memory Access. Поскольку WASM не имеет прямого доступа к физ. памяти, ядро должно предоставлять специальные буферы. Верификация: Доказать, что регион DMA памяти "закреплен" (pinned) и не может быть перемещен или освобожден, пока устройство выполняет операцию ввода-вывода.

WP ID	Задача (80 ч/ч)	Детальное описание и Критерии верификации
WP-020	Виртуальная Файловая Система (VFS)	Описание: Компонент, объединяющий различные ФС. Верификация: Доказательство отсутствия циклов в графе директорий и корректности блокировок (rw-locks) при одновременном доступе к файлам.

4. Фаза III: Интеграция AI-Native Подсистем (AIOS)

Цель фазы: Превратить ОС из пассивного исполнителя команд в интеллектуального агента. Внедрение LLM как "ядерного ресурса" (Kernel Resource), доступного через системные вызовы. **Технологический стек:** WASI-NN, Rust-based Inference (Burn/Candle), Vector DB.

Раздел 3.1: Нейросетевой Интерфейс Ядра

В традиционных ОС ИИ — это просто приложение. В AIOS — это часть инфраструктуры.

WP ID	Задача (80 ч/ч)	Детальное описание и Значение
WP-021	Реализация хоста WASI-NN	Описание: Реализация спецификации wasi-пп в ядре. Это мост между WASM-приложениями и тензорными вычислениями. Инсайт: Это стандартизирует доступ к "мозгу" системы. Любой драйвер или приложение может запросить инференс через унифицированный API, не заботясь о том, работает ли он на CPU, GPU или NPU.
WP-022	Встроенное Векторное Хранилище (Kernel Vector Store)	Описание: Реализация легковесной векторной базы данных как системного сервиса (или привилегированного компонента). Назначение: Это "Долговременная Память" (Long Term Memory) операционной системы. Здесь хранятся логи, конфигурации, документация API и история действий

WP ID	Задача (80 ч/ч)	Детальное описание и Значение
		пользователя в виде эмбеддингов.
WP-023	Менеджер Контекстного Окна (Context Pager)	Описание: Подсистема, аналогичная менеджеру виртуальной памяти. Контекстное окно LLM ограничено (например, 128k токенов). Менеджер должен "странично" подгружать (page in/out) релевантную информацию из векторного хранилища в активный контекст модели в зависимости от текущей задачи. Инсайт: Контекст рассматривается как дефицитный ресурс, требующий планирования (Scheduling).
WP-024	Драйвер VirtIO-GPU (2D/3D)	Описание: Поддержка графического ускорения. Критично не только для GUI, но и для ускорения инференса малых моделей, если NPU недоступен. Верификация: Проверка корректности очередей команд GPU для предотвращения зависаний (hangs) графической подсистемы.
WP-025	Интеграция Inference Runtime (Candle/Mistral.rs)	Описание: Встраивание движка инференса (написанного на Rust, например, candle или mistral.rs) непосредственно в слой сервисов ядра. Оптимизация под no_std или минимальное окружение. Безопасность: Использование Rust гарантирует memory safety даже при сложной тензорной математике.

Раздел 3.2: Агентные Системные Вызовы

Создание протоколов, позволяющих программам общаться с ОС на естественном языке или через семантические интен­ты.

WP ID	Задача (80 ч/ч)	Детальное описание и Значение
WP-026	Системный вызов "Намерение" (sys_intent)	Описание: Новый тип syscall, принимающий не конкретные параметры (fd, buffer), а семантическое описание задачи (напр., "найти и прочитать конфиг сети"). Ядро использует встроенную SLM (Small Language Model) для трансляции интента в цепочку классических вызовов WASI.
WP-027	Семантический Поиск Файлов	Описание: Замена традиционного find/open на семантический поиск через векторную БД (WP-022). Файловая система индексируется нейросетью, позволяя находить файлы по содержимому ("фото с котом"), а не только по имени.
WP-028	Протокол Agent-to-Agent (A2A)	Описание: Разработка стандарта межпроцессного взаимодействия для ИИ-агентов внутри ОС. Позволяет "Агенту Планировщику" договариваться с "Агентом Сети" о приоритетах трафика. Инсайт: Это эволюция IPC для мира ИИ.
WP-029	Интеграция Model Context Protocol (MCP)	Описание: Реализация стандарта MCP, позволяющего агентам ОС безопасно подключаться к внешним источникам данных и инструментам. Это делает ОС совместимой с растущей экосистемой ИИ-инструментов.
WP-030	Слой Управления и Ограничений (Guardrails	Описание: Фильтр, перехватывающий действия

WP ID	Задача (80 ч/ч)	Детальное описание и Значение
	Layer)	ИИ. Верификация: Доказательство того, что никакой высокоуровневый интент ИИ не может нарушить низкоуровневые политики безопасности (ACL). Это "Reference Monitor", верифицированный Verus, который гарантирует, что ИИ не сможет, например, перезаписать ядро, даже если решит, что это "оптимизация".

5. Фаза IV: Архитектура Самодописывания (Autopoiesis)

Цель фазы: Замкнуть цикл разработки. Система должна уметь читать свой исходный код, модифицировать его, компилировать, верифицировать и обновлять себя. Это наиболее экспериментальная часть. **Технологический стек:** Cranelift/Winch, WASM Encoder, Verus (automated mode).

Раздел 4.1: Компилятор как Сервис

Система должна содержать средства производства кода внутри себя.

WP ID	Задача (80 ч/ч)	Детальное описание и Риски
WP-031	WASM Compiler Backend (Cranelift)	Описание: Портирование Cranelift (или более легкого winch) в виде WASM-компонента. Это позволяет ОС компилировать промежуточное представление (IR) в машинный код <i>внутри</i> самой ОС. Сложность: Высокая. Требуется адаптация под окружение ядра без доступа к файловой системе хоста.
WP-032	Rust Frontend (DSL to MIR)	Описание: Реализация упрощенного фронтенда Rust или DSL, который ИИ может легко генерировать. Этот инструмент должен транслировать код в MIR (Mid-level IR) или сразу в

WP ID	Задача (80 ч/ч)	Детальное описание и Риски
		WASM, минуя тяжеловесный LLVM.
WP-033	Сервис Автоматической Верификации (Verus Service)	Описание: Интеграция решателя SMT (например, Z3) в ОС. Когда ИИ генерирует код, этот сервис запускает верификацию. Критично: Если верификация не проходит, код отбрасывается. Это главный механизм безопасности "самописывания". ИИ может "галлюцинировать", но верификатор не пропустит некорректный код в ядро.
WP-034	Промпт-инжиниринг для Генерации Кода	Описание: Создание системных промптов и контекста (RAG), обучающих LLM писать код, совместимый с Verus (с аннотациями requires, ensures). LLM должна знать API ядра и специфику доказательств.
WP-035	Механизм Горячей Замены (Hot-Reload)	Описание: Логика безопасной замены компонента. 1. Приостановить старый компонент. 2. Сериализовать его состояние (State). 3. Инициализировать новый компонент. 4. Десериализовать состояние. 5. Переключить таблицу вызовов. Верификация: Доказать функцию миграции состояния (State Transfer Function) — данные не теряются при обновлении.

Раздел 4.2: Циклы Саморазвития

WP ID	Задача (80 ч/ч)	Детальное описание и Значение
WP-036	Телеметрия и Петля Обратной Связи	Описание: Система мониторинга производительности (cache

WP ID	Задача (80 ч/ч)	Детальное описание и Значение
		misses, lock contention). Эти данные подаются на вход ИИ-агенту "Оптимизатор". Это дает ИИ <i>причину</i> для переписывания кода.
WP-037	Пайплайн Генерации Драйверов	Описание: Сценарий: ОС обнаруживает неизвестное USB-устройство -> Агент ищет спецификацию в сети (через MCP) -> Генерирует код драйвера -> Verus верифицирует -> Драйвер загружается.
WP-038	Логика Самокоррекции (Self-Correction)	Описание: Реализация цикла "Retry". Если WP-033 (Верификация) выдает ошибку, лог ошибки скормливается обратно LLM для исправления кода. Это превращает ошибку компилятора в обучающий сигнал.
WP-039	Синтез Регрессионных Тестов	Описание: ИИ должен генерировать не только код, но и доказательства (proofs) и тесты. Разработка фреймворка, где генерация реализации и спецификации идет параллельно (Co-generation).
WP-040	Blue-Green Обновление Ядра	Описание: Механизм запуска нового и старого компонента параллельно (shadow mode) для сравнения поведения на реальных данных перед окончательным переключением.

6. Фаза V: Пользовательский Опыт и Экосистема

Цель фазы: Сделать систему пригодной для использования человеком, предоставляя интерфейсы нового поколения. **Технологический стек:** WGPU, Slint/Iced, Networking stack (Smoltcp).

Раздел 5.1: Интерфейс и Сеть

WP ID	Задача (80 ч/ч)	Детальное описание и Верификация
WP-041	Сетевой Стек User-Space (Smoltcp)	Описание: Портирование smoltcp (Rust network stack) в WASM-компонент. Верификация: Проверка переходов состояний TCP (State Machine) на соответствие RFC.
WP-042	Композитор Окон (WGPU Compositor)	Описание: Создание оконного менеджера на базе wgpu (WebGPU native). Обеспечивает аппаратное ускорение интерфейса. Безопасность памяти wgpu критична для стабильности GUI.
WP-043	Рендеринг Текста и Ввод	Описание: Интеграция ab_glyph для шрифтов и обработки ввода. Связь ввода с семантическим поиском (WP-027).
WP-044	Натурально-языковая Оболочка (NL-SH)	Описание: Замена bash. Командная строка, понимающая естественный язык ("Удали все временные файлы старше недели"). Преобразует текст в вызовы API ядра.
WP-045	Интеграция GUI Фреймворка (Slint)	Описание: Портирование легковесного Rust GUI (Slint) для работы поверх композитора. Возможность "Генеративного UI", где интерфейс рисуется ИИ на лету под задачу.

Раздел 5.2: Полировка и Безопасность

WP ID	Задача (80 ч/ч)	Детальное описание
WP-046	Криптографический Сервис (wasi-crypto)	Описание: Реализация криптографии. Верификация constant-time алгоритмов для защиты от атак по сторонним каналам (side-channel attacks).
WP-047	Secure Boot Chain Extension	Описание: Расширение WP-001. Проверка цифровых

WP ID	Задача (80 ч/ч)	Детальное описание
		подписей начальных WASM-компонентов перед загрузкой.
WP-048	Многопользовательская Система Прав	Описание: Реализация UID/GID, но управляемая через систему Capabilities. ИИ-агент должен уважать эти границы.
WP-049	Системная Наблюдаемость (Observability)	Описание: Внедрение трейсинга (OpenTelemetry) в ядро для визуализации "мыслей" ИИ и путей принятия решений планировщиком.
WP-050	Финальная стабилизация и Документация	Описание: Стресс-тестирование, стабилизация доказательств Verus (устранение "хрупких" доказательств), написание документации для разработчиков драйверов.

7. Анализ Рисков и Ресурсное Обеспечение

7.1 Оценка Трудозатрат

Общий объем работ: **50 пакетов × 80 часов = 4000 часов**. Для реализации проекта в разумные сроки (12-14 месяцев) требуется команда из **3-4 инженеров-экспертов**:

1. **Инженер-формалист (The Formalist)**: Специализация на Verus, SMT-солверах, моделях памяти. Отвечает за Фазу I.
2. **Системный архитектор (The Systems Architect)**: Специализация на Rust, WASM, драйверах, низкоуровневом программировании. Отвечает за Фазу II и V.
3. **ИИ-интегратор (The AI Integrator)**: Специализация на LLM, RAG, инференс-движках. Отвечает за Фазу III и IV.

7.2 Критические Риски (Ripple Effects)

1. **Сложность Верификации ("Verus Tax")**: Написание доказательств занимает в 2-3 раза больше времени, чем написание кода. Существует риск застревания на доказательстве сложных инвариантов (например, конкурентности в планировщике).
 - *Митигация*: Использовать гибридный подход — верифицировать только TCB, остальное изолировать в WASM.
2. **Задержки ИИ (Latency)**: Использование LLM для системных вызовов (WP-026) может внести неприемлемые задержки.
 - *Митигация*: Использовать иерархию моделей (SLM для быстрых решений, LLM для сложных), кэширование интенгов и спекулятивное выполнение.
3. **Зрелость WASM Component Model**: Стандарт все еще развивается.

- *Митигация*: Активное участие в стандартах (Bytecode Alliance) и готовность писать собственные полифилы.

7.3 Влияние на индустрию

Реализация **piOS** создаст прецедент "Software 2.0" операционной системы.

- **Безопасность**: Устранение целых классов уязвимостей (memory safety, logic bugs) через верификацию и изоляцию.
- **Экономика**: Снижение стоимости поддержки драйверов благодаря автогенерации.
- **Moving Target Defense**: Возможность ОС постоянно перекомпилировать свои компоненты с рандомизацией памяти делает эксплуатацию уязвимостей (ROP/JOP) практически невозможной.

8. Заключение

Предложенный план описывает амбициозный, но технически реализуемый путь к созданию операционной системы, которая является не просто набором утилит, а живым, адаптирующимся организмом. Сочетание математической строгости Verus с креативностью генеративного ИИ создает уникальную архитектуру, где надежность ядра гарантирует, что саморазвитие системы никогда не приведет к самоуничтожению. Это следующий логический шаг в эволюции системной инженерии.

Ссылки на использованные материалы

Интегрированы в текст отчета:....

Источники

1. verus-lang/verus: Verified Rust for low-level systems code - GitHub, <https://github.com/verus-lang/verus> 2. Verus: A Practical Foundation for Systems Verification - Tej Chajed, <https://www.chajed.io/papers/verus:sosp2024.pdf> 3. Asterinas: A Linux ABI-Compatible, Rust-Based Framekernel OS with a Small and Sound TCB - arXiv, <https://arxiv.org/html/2506.03876v1> 4. The WebAssembly Component Model: Introduction, <https://component-model.bytecodealliance.org/> 5. What is the WebAssembly Component Model? - F5, <https://www.f5.com/company/blog/what-is-the-webassembly-component-model> 6. LLM as OS, Agents as Apps: Envisioning AIOS, Agents and the AIOS-Agent Ecosystem - arXiv, <https://arxiv.org/html/2312.03815v2> 7. agiresearch/AIOS: AIOS: AI Agent Operating System - GitHub, <https://github.com/agiresearch/AIOS> 8. Building self-evolving AI systems: exploring the architecture | by Pavel Buchnev | Jan, 2026, <https://butschster.medium.com/building-self-evolving-ai-systems-exploring-the-architecture-a63912fd72c4> 9. Hyperlight Wasm: Fast, secure, and OS-free - Microsoft Open Source Blog, <https://opensource.microsoft.com/blog/2025/03/26/hyperlight-wasm-fast-secure-and-os-free> 10. Theseus is a new OS written from scratch in Rust to experiment with novel OS structure, better state management, and how to shift OS responsibilities like resource management into the compiler. - Reddit, https://www.reddit.com/r/rust/comments/jpfuwe/theseus_is_a_new_os_written_from_scratch_in_rust/